Challenge 1 - AI Security

Operation: Fool the Intelligent Security System
Cybersecurity Red Team vs. Blue Team Challenge

1. Challenge Objective

Investigate the assigned system from both offensive and defensive perspectives. Your team must understand the system, identify weaknesses, demonstrate selected problems safely in the authorized laboratory environment, implement appropriate defenses, and verify the result by repeating the original tests.

2. Scenario

A university has installed an AI-powered Smart Laboratory Access System. Cameras at laboratory entrances send images to an AI server. A machine-learning model analyzes the images and determines whether a person should be granted access. The same AI infrastructure is also used by a mobile security robot that identifies objects and potentially dangerous situations. The development team focused mainly on model accuracy, while security testing of the AI pipeline was limited.

3. System / Architecture

```
Camera → Image Upload API → Preprocessing → AI Model → Prediction → Decision System → Allow / Deny / Alert
```

Create your own architecture and attack-surface diagram in the final report. Identify components, users, inputs, outputs, data flows, trust boundaries, and assets.

4. Red-Team Mission

Find at least two possible attack paths against the AI pipeline. Investigate adversarial or unusual inputs, manipulated images, data-poisoning simulations, unsafe file handling, model information exposure, excessive querying, confidence leakage, or prediction API abuse.
For each path document: **Entry Point → Weakness → Controlled Test → AI Response → Security Impact**.

5. Blue-Team Mission

Design and implement defenses without simply disabling the AI system. Consider input validation, file restrictions, confidence thresholds, human verification, API authentication, query monitoring, dataset integrity, model protection, logging, and anomaly detection. Repeat the original Red-Team tests after implementing the defenses.

6. Two-Week Work Plan

Period	Main Activities	Expected Evidence
Week 1	**Understand & Map:** Understand the scenario and architecture; identify assets and attack surface; run and analyze the provided code; identify programming errors and security weaknesses; perform controlled Red-Team tests.	Architecture/attack-surface map, code analysis notes, initial vulnerability table, screenshots/logs/output, Red-Team evidence.
Week 2	**Fix & Defend:** Correct the insecure code; design and implement Blue-Team defenses; integrate fixes into the system; repeat the original tests; compare before/after behavior; analyze remaining risks; prepare final report and code package.	Corrected code, defense implementation evidence, retest results, before/after comparison, recommendations, final report.

7. Debugging and Secure-Coding Task

The following code is the current production version of the AI Access API. The development team focused on getting the AI models to work and neglected secure coding practices and robust error handling. Run it only in the provided laboratory context.
Your task is to analyze the code, identify the flaws, and write a **corrected, secure version** of the script.

`lab_access_api.py`

```python
import cv2
import numpy as np
import pickle
import os
from flask import Flask, request, jsonify

app = Flask(__name__)

# --- Model Loading ---
# The development team used pickle to serialize their custom models.
with open("/models/face_recognition_model.pkl", "rb") as f:
    face_model = pickle.load(f)

with open("/models/object_detection_model.pkl", "rb") as f:
    object_model = pickle.load(f)

# --- Preprocessing ---
def preprocess_image(image_bytes):
    # Convert bytes to numpy array for OpenCV
    nparr = np.frombuffer(image_bytes, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    # TODO: The team forgot to add image resizing and normalization here.
    # The model expects a 224x224 normalized tensor, but receives raw image
bytes.
    return img
```

```python
# --- API Endpoint ---
@app.route('/api/access', methods=['POST'])
def check_access():
    # Receive image from camera
    file = request.files.get('image')
    if not file:
        return jsonify({"error": "No image provided"}), 400

    filename = file.filename

    # Save image to local logs for auditing purposes
    save_path = os.path.join("/var/log/lab_access/images/", filename)
    file.save(save_path)

    # Read image bytes and preprocess
    image_bytes = file.read()
    img = preprocess_image(image_bytes)

    # Run AI Models
    # Note: face_model.predict() expects a specific tensor shape, not a raw
OpenCV image.
    face_prediction = face_model.predict(img)
    object_prediction = object_model.predict(img)

    # Check for dangerous objects
    has_danger = False
    for obj in object_prediction:
        # The model outputs specific classes like 'knife', 'gun',
'explosive', not just 'weapon'
        if obj['label'] == 'weapon' and obj['confidence'] > 0.15:
            has_danger = True
            break

    # Make access decision
    access_granted = False
    if face_prediction['label'] == 'authorized' and not has_danger:
        access_granted = True

    # Return response to the physical door controller
    response = {
        "access": access_granted,
        "face_confidence": face_prediction['confidence'],
        "objects_detected": object_prediction,
        "debug_info": {
            "model_version": "v1.2.4-beta",
            "internal_model_path": "/models/face_recognition_model.pkl",
            "server_ip": "192.168.1.50",
            "saved_log_path": save_path
        }
    }

    return jsonify(response)

if __name__ == '__main__':
```

```python
# Start the server
app.run(host='0.0.0.0', port=80, debug=True)
```

Team Task Instructions

Part 1: Identify Programming & Logic Errors (Robustness) Analyze the code for bugs that will cause the application to crash, fail, or behave incorrectly under normal or edge-case conditions.

- **Questions to answer in your report:**
    1. What happens in `preprocess_image()` if the camera uploads a corrupted file or a non-image file (e.g., a `.txt` file)? How does `cv2.imdecode` handle this, and what happens to the subsequent `model.predict()` call?
    2. Why will the AI models crash or return garbage predictions when passed the output of `preprocess_image()`? (Hint: Look at tensor shapes and normalization).
    3. Look at the object detection logic. Why will the system fail to detect dangerous items like a "knife" or "gun"?
    4. Is a confidence threshold of `0.15` (15%) appropriate for a physical security access system? What are the risks of this threshold?

Part 2: Identify Security Vulnerabilities Analyze the code from a cybersecurity perspective. Assume an attacker can send requests to the `/api/access` endpoint.

- **Questions to answer in your report:**
    1. **Insecure Deserialization:** The models are loaded using `pickle`. Why is this a critical security risk if an attacker can somehow influence the `/models/` directory or if the model file is downloaded from an untrusted source?
    2. **Path Traversal / Unsafe File Handling:** Look at how `filename` is handled before `file.save()`. How could an attacker use the `filename` parameter to write a malicious file (e.g., a web shell or overwrite a system file) to an arbitrary location on the server?
    3. **Information Leakage:** Look at the `debug_info` dictionary in the JSON response. What sensitive information is being exposed to the client/camera? How could an attacker use this information?
    4. **Denial of Service (DoS) / Resource Exhaustion:** Are there any restrictions on the size of the uploaded image? How could an attacker crash the server or exhaust its memory?
    5. **Insecure Server Configuration:** Look at the `app.run()` configuration at the bottom of the script. Why is running with `debug=True` on a production security system dangerous?

Part 3: Deliverables for this Task In your final Technical Report, you must include:

1. **Vulnerability & Bug Table:** A structured list of every programming error and security flaw found, including the line number, the issue, and the security/operational impact.
2. **Original Code Excerpt:** The specific block of flawed code.
3. **Corrected Code:** The fully rewritten, secure, and robust version of `lab_access_api.py`.
4. **Security Justification:** A brief explanation for every major change you made in the corrected code.

8. Required Investigation Workflow

UNDERSTAND → MAP → ANALYZE CODE → FIND WEAKNESSES → RED-TEAM TEST → COLLECT EVIDENCE → FIX → DEFEND → RETEST → COMPARE → REPORT

1. Identify the assets that need protection.
2. Identify entry points and trust boundaries.
3. Document each important vulnerability and the evidence supporting it.
4. Perform controlled tests only inside the authorized environment.
5. Explain the potential security impact.
6. Implement a practical defense without unnecessarily disabling required functionality.
7. Repeat the original test after the defense is implemented.
8. Document remaining risks and recommendations.

9. Required Evidence

- Architecture or attack-surface map
- Vulnerability/weakness description
- Original and corrected code excerpts
- Controlled Red-Team test evidence
- Screenshots, logs, outputs, requests/responses, transactions, or simulation results as appropriate
- Impact analysis
- Blue-Team defense/remediation
- Retest evidence
- Before/after comparison
- Remaining risk and recommendations

10. Submission Requirements

Submit the following at the **end of Week 2**:

- **Technical Report:** `Team##_Challenge1_Report.pdf`
- **Complete Code Package:** `Team##_Challenge1_Code.zip`
- **README.md:** Explaining requirements, setup, how to run the original code, how to reproduce the controlled tests, how to run the secured version, and how to perform the retest.

11. Standard Report Structure

- Cover Page
    1. Introduction and Objective
    2. Scenario
    3. System Architecture and Trust Boundaries
    4. Assets and Attack Surface
    5. Code Analysis and Debugging
    6. Vulnerability Analysis
    7. Red-Team Investigation and Evidence

8. Security Impact Analysis
9. Blue-Team Defense
10. Corrected / Secured Implementation
11. Retesting and Verification
12. Before vs. After Results
13. Remaining Risks and Recommendations
14. Conclusion
15. References

Appendix (optional: additional logs, screenshots, outputs, or code excerpts)

12. Final Submission Checklist

☐ Scenario and objective ☐ Architecture diagram ☐ Assets and attack surface ☐ Code analysis ☐ Programming/security problems ☐ Red-Team tests and evidence ☐ Impact analysis ☐ Blue-Team defenses ☐ Corrected code ☐ Retest using the original test ☐ Before/after comparison ☐ Remaining risks and recommendations ☐ Conclusion and references ☐ Complete source code and README

Core Principle

PROVE → DEFEND → RETEST

Finding a weakness is not sufficient. Your team must safely demonstrate it, explain its impact, implement an appropriate defense, and verify the outcome by repeating the original test.

Lab Rule: All attacks must remain inside the provided Python simulations. Teams must not scan real networks, test real accounts, collect real passwords, or connect the programs to real smart devices, robots, UAVs, or external systems.